

PROJET DE FIN DE MODULE

Machine Learning

Phase 2

EDA et préparation des données

1. Objectifs de la phase

À l'issue de cette phase, vous devez avoir produit un dataset prêt pour la modélisation, accompagné d'une compréhension approfondie de ses caractéristiques. Concrètement, vous devez avoir :

1. Mené une analyse exploratoire (EDA) approfondie du dataset constitué en Phase 1.
2. Nettoyé les données (valeurs manquantes, doublons, incohérences, outliers).
3. Transformé les variables (encodage, normalisation, ...).
4. Créé au moins 2 nouvelles variables dérivées (feature engineering) justifiées métier.
5. Construit un pipeline de preprocessing reproductible et sérialisable.
6. Préparé une stratégie de gestion du déséquilibre, prête à être testée en Phase 3.

2. Travail à réaliser

2.1. Analyse exploratoire des données (EDA)

a) Exploration univariée

Pour chaque variable du dataset, vous devez produire :

- **Variables numériques** : histogramme, boxplot, statistiques descriptives (moyenne, médiane, écart-type, quartiles, min/max).
- **Variables catégorielles** : diagramme en barres des fréquences, identification des modalités rares (< 1 %).

b) Exploration bivariée

L'objectif est d'identifier les variables potentiellement prédictives de la cible :

- **Numérique × cible** : boxplot par classe.
- **Catégorielle × cible** : tableau croisé, taux de classe positive par modalité.
- **Numérique × numérique** : matrice de corrélation, heatmap, identification des features fortement corrélées entre elles.

c) Analyse spécifique du déséquilibre

Cette analyse est obligatoire et structurante pour la suite :

- Ratio exact des classes (ex : 87 % / 13 %).
- Visualisation de la distribution (camembert ou diagramme en barres).
- Statistiques descriptives séparées pour chaque classe : pour identifier les features dont la distribution diffère significativement entre classes minoritaire et majoritaire.
- Identification des « features signaux » : variables où la classe minoritaire se comporte clairement différemment de la majoritaire — ce sont vos meilleurs prédicteurs potentiels.

Livrable de cette sous-étape

Notebook **02_eda.ipynb** structuré, commenté, avec une synthèse en fin de notebook.

2.2. Nettoyage des données

a) Valeurs manquantes

Pour chaque variable contenant des valeurs manquantes, vous devez :

1. Quantifier le taux de valeurs manquantes.
2. Choisir une stratégie justifiée : suppression de la ligne (si $< 5\%$), suppression de la colonne (si $> 50\%$ ou peu informative), imputation simple (moyenne, médiane, mode) ou imputation par modèle (KNN imputer, ...).
3. Documenter chaque choix dans un tableau récapitulatif.

b) Doublons et incohérences

- Détection et suppression des doublons exacts.
- Vérification des incohérences logiques (date de fin antérieure à la date de début, valeurs négatives sur des grandeurs positives, etc.).
- Vérification de la cohérence métier (ex : un film de 1920 ne peut pas avoir un budget de 200 M\$).

c) Outliers

Pour chaque variable numérique, identifiez les outliers via :

- Méthode IQR ($1,5 \times$ écart interquartile) ou méthode du Z-score ($|z| > 3$).
- Visualisation par boxplot.

Pour chaque outlier identifié, choisissez : conservation (s'ils sont valides et porteurs de signal) ou suppression (s'ils sont des erreurs).

Attention au data leakage

Le nettoyage doit s'effectuer **AVANT** la séparation train/test (ou de manière identique sur les deux), mais les **statistiques utilisées pour l'imputation (moyenne, médiane) doivent être calculées UNIQUEMENT sur le jeu d'entraînement** puis appliquées au jeu de test. Sinon, c'est un data leakage qui sera pénalisé.

2.3. Transformation des variables

a) Encodage des variables catégorielles

Choisissez la technique adaptée à chaque variable :

Technique	Quand l'utiliser	Précautions
One-Hot Encoding	Variables nominales avec faible cardinalité (< 15 modalités)	Explosion dimensionnelle si cardinalité élevée
Ordinal Encoding	Variables ordinales avec ordre naturel (ex : taille S/M/L)	Ne jamais l'utiliser pour des variables nominales

b) Normalisation des variables numériques

- **StandardScaler** (moyenne 0, écart-type 1) : recommandé pour la régression logistique, SVM, KNN, réseaux de neurones.
- **MinMaxScaler** (intervalle [0,1]) : recommandé quand on veut préserver la distribution originale.
- **RobustScaler** (basé sur médiane et IQR) : recommandé en présence d'outliers.
- **Pas de scaling nécessaire** pour les arbres et leurs ensembles (Random Forest, Gradient Boosting) : ils sont insensibles à l'échelle.

2.4. Feature engineering

Vous devez créer au minimum 2 nouvelles variables dérivées, justifiées métier. C'est l'étape qui peut faire la différence entre un projet médiocre et un projet excellent.

Catégories de features à explorer

- **Features temporelles** : extraction depuis une date (jour de la semaine, mois, saison, weekend/jour ouvré, jour férié), durée écoulée depuis un événement (âge du compte, ancienneté de la dernière interaction).
- **Features d'agrégation** : moyenne / somme / max / min / écart-type calculés sur un sous-groupe (ex : nombre moyen de likes des posts précédents d'un même auteur).
- **Ratios et différences** : rapport entre deux variables (budget marketing / budget production), différence (prix actuel – prix moyen).
- **Features d'interaction** : produit ou combinaison de deux variables qui ont du sens ensemble (genre × saison de sortie pour un film).
- **Features de comptage** : nombre d'acteurs au casting, nombre de tags.
- **Binning (discrétisation)** : transformer une variable continue en catégories (ex : tranche d'âge, gamme de prix).

2.5. Construction du pipeline de preprocessing

Un pipeline scikit-learn (`sklearn.pipeline.Pipeline` ou `ColumnTransformer`) est obligatoire. Ses bénéfices :

- **Reproductibilité** : les mêmes transformations seront appliquées en Phase 4 (déploiement) qu'en Phase 3 (entraînement).
- **Sérialisation** : le pipeline complet (preprocessing + modèle) peut être sauvegardé en un seul fichier `.joblib`.
- **Compatibilité avec GridSearchCV** : les hyperparamètres du preprocessing peuvent eux-mêmes être tunés (ajustés).

Structure attendue du pipeline

ColumnTransformer avec : un sous-pipeline pour les variables numériques (imputer + scaler), un sous-pipeline pour les variables catégorielles (imputer + encoder), et la conservation des variables qui n'ont pas besoin de transformation. L'ensemble doit être encapsulé dans un **Pipeline** final auquel on ajoutera le modèle en Phase 3.

2.6. Séparation train / validation / test

La séparation se fait après le feature engineering (qui ne dépend pas de la cible) et avant tout entraînement. Vous devez :

- **Stratifier sur la cible** pour préserver le ratio de déséquilibre dans chaque sous-ensemble (`stratify=y`).
- **Fixer un `random_state`** pour la reproductibilité.
- **Choisir une proportion adaptée** : typiquement 60 / 20 / 20 ou 70 / 15 / 15. Le test set ne sera utilisé qu'une seule fois en fin de Phase 3.

2.7. Préparation de la stratégie de gestion du déséquilibre

Vous ne testez pas encore les modèles, mais vous préparez les techniques que vous comparerez en Phase 3. Au minimum 3 stratégies à implémenter :

- **Aucun rééquilibrage** (baseline) avec `class_weight='balanced'` sur les modèles qui le supportent.
- **Oversampling de la minoritaire** : SMOTE, ADASYN, ou `RandomOverSampler` (via la librairie `imbalanced-learn`).
- **Undersampling de la majoritaire** : `RandomUnderSampler`, Tomek Links, ou `NearMiss`.
- **(Bonus) Combinaison** : `SMOTETomek` ou `SMOTEENN`, qui combinent oversampling et nettoyage/undersampling.

Important : ces techniques doivent être intégrées dans le pipeline via `imblearn.pipeline.Pipeline` (et non `sklearn.pipeline.Pipeline`) pour s'appliquer correctement **uniquement à l'entraînement**.

3. Livrables de fin de phase

N°	Livrable	Contenu attendu
1	Notebook EDA (notebooks/02_eda.ipynb)	Exploration univariée et bivariée, analyse spécifique du déséquilibre, synthèse.
2	Notebook de préparation (notebooks/03_preprocessing.ipynb)	Nettoyage, transformations, feature engineering, construction du pipeline, séparation train/validation/test stratifiée.
3	Pipeline sérialisé (models/preprocessor.joblib)	Pipeline scikit-learn complet, prêt à être combiné avec un modèle en Phase 3.
4	Datasets traités (data/processed/)	Fichiers train.csv, validation.csv, test.csv issus du split stratifié.
5	Tableau de décision (preprocessing_decisions.md)	Pour chaque variable : action effectuée (suppression, imputation, encodage, transformation) et justification.

PROJET DE FIN DE MODULE

Machine Learning

Phase 3

Modélisation, tuning et évaluation

1. Objectifs de la phase

À l'issue de cette phase, vous devez avoir identifié, optimisé et évalué le meilleur modèle pour votre problème, en tenant compte rigoureusement du déséquilibre et du coût métier asymétrique défini en Phase 1. Concrètement, vous devez avoir :

1. Entraîné au moins 4 types de modèles différents.
2. Comparé au moins 3 stratégies de gestion du déséquilibre.
3. Optimisé les hyperparamètres du meilleur modèle.
4. Évalué le modèle final sur le jeu de test, avec les métriques définies en Phase 1.
5. Ajusté le seuil de décision selon le coût métier.

2. Travail à réaliser

2.1. Entraînement de plusieurs familles de modèles

Vous devez tester au minimum 4 types de modèles, choisis pour leur diversité algorithmique. Les choix recommandés pour un problème de classification déséquilibrée :

Modèle	Forces	Faiblesses
Régression logistique (baseline)	Rapide, interprétable, baseline solide, supporte class_weight	Suppose la linéarité, sensible aux outliers
Arbre de décision	Très interprétable, capture les non-linéarités	Sur-apprentissage facile, instable
Random Forest	Robuste, peu de tuning nécessaire, gère le déséquilibre	Plus lent, moins interprétable qu'un arbre simple
Gradient Boosting (XGBoost / LightGBM)	Souvent le meilleur sur tabulaire, scale_pos_weight intégré	Tuning délicat, risque de sur-apprentissage
SVM	Efficace en haute dimension	Très lent sur > 10 000 lignes
Réseau de neurones multicouches (MLP)	Capture des relations très complexes et non linéaires.	Boîte noire (interprétation difficile), nécessite plus de données et de tuning

Protocole d'entraînement

- Pour chaque modèle, utiliser le pipeline construit en Phase 2 (preprocessing + modèle).
- Validation croisée stratifiée à 5 splits (StratifiedKFold(n_splits=5)) sur le jeu d'entraînement.
- Reporter la moyenne ET l'écart-type (σ) de la métrique principale (e.g., F1).
- Fixer le random_state partout pour la reproductibilité.

2.2. Comparaison des stratégies de gestion du déséquilibre

Pour chacun des 4 modèles, comparer au minimum 3 stratégies, soit 12 configurations à évaluer :

Modèle	Sans réeq. (class_weight)	SMOTE	Undersampling
Modèle 1 (Logistic Reg)	$F1 \pm \sigma$	$F1 \pm \sigma$	$F1 \pm \sigma$
Modèle 2 (Decision Tree)	$F1 \pm \sigma$	$F1 \pm \sigma$	$F1 \pm \sigma$
Modèle 3 (XGBoost)	$F1 \pm \sigma$	$F1 \pm \sigma$	$F1 \pm \sigma$
Modèle 4 (MLP)	$F1 \pm \sigma$	$F1 \pm \sigma$	$F1 \pm \sigma$

Sélectionnez la combinaison « modèle × stratégie de rééquilibrage » qui maximise la métrique principale. C'est sur cette combinaison que portera le tuning des hyperparamètres.

2.3. Optimisation des hyperparamètres

Méthodes recommandées

- **GridSearchCV** : exhaustive sur une grille restreinte. À privilégier si vous avez peu d'hyperparamètres et du temps de calcul.
- **RandomizedSearchCV** : exploration aléatoire sur des distributions. Plus efficace quand l'espace est grand.

Hyperparamètres à tuner par modèle

- **Régression logistique** : C (régularisation : 0.01, 0.1, 1, 10), penalty (l2 par défaut ou l1), et class_weight='balanced' pour gérer le déséquilibre. Si vous avez un avertissement de non-convergence, augmentez max_iter à 5000, solver (liblinear pour petits datasets et l1/l2, saga pour gros datasets et elasticnet, lbfgs par défaut pour l2).
- **Decision Tree** : max_depth (3, 5, 7, 10, pour éviter le sur-apprentissage), min_samples_leaf (1, 5, 10, 20), et class_weight='balanced'.
- **Random Forest** : n_estimators (200 puis 500), max_depth (10, 20, None), min_samples_leaf (1, 2, 5), max_features ('sqrt' est presque toujours optimal), et class_weight='balanced_subsample' qui surpasse souvent SMOTE pour ce modèle.
- **XGBoost** : n_estimators, max_depth, learning_rate, subsample, colsample_bytree, scale_pos_weight.
- max_depth (3, 5, 7), learning_rate (0.01, 0.05, 0.1), n_estimators (300 à 1000, idéalement avec early_stopping_rounds=50), subsample et colsample_bytree (0.7, 0.8, 1.0), et surtout scale_pos_weight = $\frac{\sum(y==0)}{\sum(y==1)}$ pour le déséquilibre.
- **SVM** : C (paramètre de régularisation : 0.01, 0.1, 1, 10, 100), kernel (linear, rbf, poly, sigmoid), gamma (coefficient du noyau pour rbf/poly/sigmoid : 'scale', 'auto', ou valeurs explicites

comme 0.001, 0.01, 0.1, 1), `class_weight` ('balanced' ou dictionnaire de poids pour gérer le déséquilibre), `probability=True` (obligatoire pour obtenir des probabilités via `predict_proba`, nécessaire au calcul du PR-AUC et à l'optimisation du seuil).

- **MLP (Multi-Layer Perceptron)** : `hidden_layer_sizes` (architecture des couches cachées, ex. (128, 64, 32)), `activation` (relu, tanh, logistic), `solver` (adam pour grands datasets, lbfgs pour petits datasets), `alpha` (régularisation L2 : 0.0001, 0.001, 0.01), `learning_rate_init` (0.0001, 0.001, 0.01), `batch_size` (32, 64, 128, ou auto), `max_iter` (200, 500, 1000), `early_stopping` (True pour éviter le sur-apprentissage).

Bonne pratique

Justifiez vos plages de recherche dans le notebook (commentaires expliquant pourquoi vous testez `max_depth` $\in [3, 5, 7, 10]$ plutôt que $[1, 50]$). Une grille mal choisie est aussi pénalisante qu'une absence de tuning.

2.4. Évaluation finale sur le jeu de test

Une fois le meilleur modèle sélectionné et tuné, vous l'évaluez UNE SEULE FOIS sur le jeu de test (intouché jusqu'ici). Reporter :

Métriques quantitatives

- Métrique principale définie en Phase 1 (e.g., F1), comparée au seuil de succès fixé.
- Métriques complémentaires : précision, rappel, accuracy, etc..
- Matrice de confusion avec interprétation métier des 4 cellules (TP, TN, FP, FN).

Visualisations

- Courbe ROC.
- Courbe Precision-Recall (la plus importante en contexte déséquilibré).
- Distribution des probabilités prédites par classe.

Comparaison au seuil de succès

Indiquez explicitement : les objectifs ML fixés en Phase 1 sont-ils atteints ? Si non, expliquez pourquoi (signal insuffisant, déséquilibre trop fort, features mal choisies, etc.).

2.5. Optimisation du seuil de décision (J11)

Le seuil de décision par défaut (0,5) est **rarement optimal** sur des données déséquilibrées. Vous devez :

1. Tracer precision et recall en fonction du seuil (de 0,1 à 0,9 par pas de 0,01).
2. Calculer le coût total prédit pour chaque seuil, en utilisant la matrice de coût asymétrique définie en Phase 1.
3. Identifier le seuil qui minimise le coût métier total.
4. Justifier ce choix de seuil par rapport au seuil par défaut.
5. Reporter la matrice de confusion au seuil optimal et au seuil par défaut, pour montrer le gain.

Exemple de calcul de coût

Si un faux négatif coûte 1 000 € (fraude non détectée) et un faux positif coûte 50 € (transaction légitime bloquée), alors : **Coût total** = 1 000 × FN + 50 × FP. Le seuil optimal est celui qui minimise cette somme sur le jeu de validation.

3. Livrables de fin de phase

N°	Livrable	Contenu attendu
1	Notebook de modélisation (notebooks/04_modeling.ipynb)	Entraînement des 4 modèles, comparaison des stratégies de rééquilibrage, tableau comparatif (12 configurations).
2	Notebook de tuning (notebooks/05_tuning.ipynb)	Optimisation des hyperparamètres du meilleur modèle, justification de la grille de recherche.
3	Notebook d'évaluation (notebooks/06_evaluation.ipynb)	Évaluation finale sur le test set, optimisation du seuil avec matrice de coût, comparaison aux objectifs ML de la Phase 1.
4	Modèle final sérialisé (models/final_model.joblib)	Pipeline complet (preprocessing + modèle tuné), avec le seuil de décision optimal en métadonnée.

PROJET DE FIN DE MODULE

Machine Learning

Phase 4

Déploiement et livraison finale

1. Objectifs de la phase

Cette dernière phase transforme votre travail de recherche en un produit utilisable. À l'issue, vous devez avoir :

1. Construit une API REST exposant le modèle final.
2. Développé une interface utilisateur permettant à un non-data-scientist d'utiliser le modèle.
3. Conteneurisé l'ensemble avec Docker pour garantir la portabilité (optionnel)
4. Documenté complètement le projet.
5. Rédigé le rapport final.

2. Travail à réaliser

2.1. Construction de l'API REST avec FastAPI

FastAPI est obligatoire. Il offre la documentation Swagger automatique, la validation Pydantic, et des performances élevées.

Endpoints minimaux requis

Endpoint	Méthode	Description
/	GET	Page d'accueil avec informations sur l'API et lien vers la doc Swagger.
/health	GET	Vérifie que l'API et le modèle sont opérationnels (retourne 200 OK).
/model/info	GET	Métadonnées du modèle : type, version, date d'entraînement, métriques de performance, seuil utilisé.
/predict	POST	Prédiction unitaire : reçoit les features d'un cas, retourne classe + probabilité.
/predict/batch	POST	Prédiction par lot : reçoit un fichier CSV, retourne un fichier CSV enrichi des prédictions.

Schéma de réponse de /predict (exemple)

L'API doit retourner une réponse JSON structurée incluant la prédiction, la probabilité brute et le seuil appliqué. Exemple :

```
{
  "prediction": "echec",
  "probability": 0.78,
  "threshold": 0.42,
  "confidence": "high"
}
```

Bonnes pratiques

- Validation des entrées avec Pydantic (BaseModel) — types, plages de valeurs, valeurs autorisées.
- Gestion des erreurs avec codes HTTP appropriés (400 si entrée invalide, 500 si erreur serveur).
- Logging des requêtes pour traçabilité.
- Documentation Swagger enrichie (descriptions, exemples).
- Chargement du modèle UNE SEULE FOIS au démarrage (au lieu de le recharger à chaque requête).

2.2. Interface utilisateur

Vous avez le choix entre Streamlit (recommandé) ou Gradio. L'interface doit être utilisable par un non-technicien.

Composants requis

- **Formulaire de saisie** : tous les inputs nécessaires à la prédiction, avec validation côté UI (champs obligatoires, plages, listes déroulantes pour les variables catégorielles).
- **Affichage de la prédiction** : résultat clair (« risque élevé » / « risque faible »), pas juste une valeur 0/1.
- **Mode batch** : permettre l'upload d'un CSV pour traitement en lot.
- **Page d'information** : expliquer en quelques phrases ce que fait le modèle, sa performance, ses limites.

2.3. Conteneurisation Docker (optionel)

Fichiers obligatoires

- **Dockerfile** : image basée sur python:3.11-slim, copie du code et du modèle, installation des dépendances, exposition du port, commande de démarrage.
- **.dockerignore** : exclure data brutes, notebooks, .git, venv, __pycache__, .env.
- **docker-compose.yml** : si vous séparez l'API et l'UI en deux services. Définir le réseau, les ports, les volumes pour les modèles.
- **requirements.txt** : versions figées de toutes les dépendances (fastapi==0.115.0, scikit-learn==1.5.2, etc.).

Critères de qualité du Dockerfile

- Image finale légère (< 1 Go idéalement).
- Pas de fichiers inutiles dans l'image (data/, notebooks/).
- Utilisation de couches Docker optimisées (requirements installés avant le code applicatif pour bénéficier du cache).

- Variable d'environnement pour la configuration (port, niveau de log, chemin du modèle).
- Healthcheck défini dans le docker-compose.

Test obligatoire avant rendu

Lancer **docker-compose up** sur une machine vierge (autre que la vôtre) doit suffire à démarrer l'application complète, accessible via un navigateur.

2.4. Documentation finale

README.md (à la racine du dépôt)

- Titre et description du projet en 2-3 phrases.
- Captures d'écran de l'interface (au moins 2).
- Instructions d'installation (avec et sans Docker).
- Exemple d'utilisation : commande curl pour tester l'API, capture du flow utilisateur.
- Architecture du dépôt (arborescence commentée).
- Lien vers la documentation Swagger.
- Limites connues du modèle.
- Licence.

Rapport final

Structure CRISP-DM enrichie :

1. Introduction et contexte métier (Phase 1).
2. Description et compréhension des données (Phase 1).
3. Préparation des données (Phase 2).
4. Modélisation, tuning et évaluation (Phase 3).
5. Interprétation et recommandations métier (Phase 3).
6. Architecture et déploiement (Phase 4).
7. Conclusion, limites, perspectives.
8. Annexes (figures complémentaires, code clé).

3. Livrables finaux du projet

N°	Livable	Contenu attendu
1	Dépôt Git complet	Code source modulaire (src/, app/, tests/), notebooks numérotés, données traitées, modèles sérialisés, Dockerfile, docker-compose.yml, README.md, requirements.txt.
2	Application déployée localement	Démarrable via docker-compose up. API accessible sur un port défini, UI accessible via navigateur.
3	Rapport final (report.pdf)	Structure CRISP-DM, figures et tableaux légendés, bibliographie.
4	Support de présentation (slides.ppt)	Slides présentant l'essentiel du projet.
5	Démo live	Application fonctionnelle présentée en direct le jour de la présentation.

Bon courage à toutes et à tous pour ce projet — il est l'occasion de mobiliser et de démontrer toutes les compétences acquises au cours du module.